

TPSIT

Codifiche Errore

Prof. Della Vecchia P.

Codici di rilevazione dell'errore

- Nello scambio di informazioni binario tra i bus di dati e collegamenti vari sappiamo che i dati per via di **disturbi del segnale**, dovuto anche ad alterazioni elettromagnetiche può subire danneggiamenti o distorsioni.
- Per evitare ciò, entrano in gioco le codifiche per la **rilevazione dell'errore** che ci aiutano a garantire l'avvenuto inoltro e ricezione corretto dei dati.

CODICI PESATI e non pesati

Tra le altre codifiche digitali a **lunghezza fissa** vi sono:

Codici Pesati e Codici non pesati

- Nei **Pesati** ogni cifra ha un certo **valore specifico (peso)** basato sulla **sua posizione**.

Esempi di codici pesati:

Codifica Aiken

- Nei codici **non pesati** invece il valore dipende da **regole logiche o sequenziali**.

Esempi di codici non pesati:

Codifica Gray

Codice di hamming

CODICI PESATI – Codifica Aiken

$$Y = 9^1 - x$$

AIKEN – COMPLEMENTO A 9

$$9 - 5 = 4$$

- Il complemento a N di un certo numero x è quel numero ottiene con $N - x$

La codifica **Aiken** in elettronica usa il completamento a 9 per ricavare il loro valore binario, quindi numeri da 0 a 9

Si usa un nibble di bit -> 0010

Per i numeri < di 5, il nibble usa il **binario classico**

Per i **numeri >= di 5** il passaggio è il seguente

$CA_9[5]$ è 4 **Proprietà di Aiken** – mi creo il nibble

$CA_9[5]$ -> 0100 -> **inverto i bit (complemento a 1)** -> 1011 quindi in Aiken il 5 corrisponde a 1011

$CA_9[6]$ è 3 $CA_9[6]$ -> 0011 -> **inverto i bit (complemento a 1)** -> 1100

quindi il 6 corrisponde a 1100

Table I. Decimal Values and Associated 4-bit Aiken Code and Standard Binary Code

Decimal Digit	Aiken (2421) Code	Binary (8421) Code
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	1011	0101
6	1100	0110
7	1101	0111
8	1110	1000
9	1111	1001

CODICI NON PESATI – Codifica Gray

Xor nidificati

Serve a evitare errori quando i numeri cambiano in sistemi digitali.

- ① In binario normale, tra due numeri consecutivi **possono cambiare più bit** contemporaneamente. Esempio: da 3 a 4 (3 = 011, 4 = 100) → cambiano tutti e 3 i bit.

Se stai leggendo quei bit con un circuito, alcuni **bit possono essere letti prima degli altri**, creando **valori intermedi sbagliati**.

- ② La soluzione: codice Gray. In Gray, **solo un bit cambia tra numeri consecutivi**.

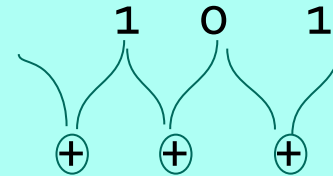
Questo significa che anche se c'è un ritardo nella lettura, il numero letto non salta mai di più di 1.

Binario: 101

somme xor: XOR tra bit corrente e precedente del binario

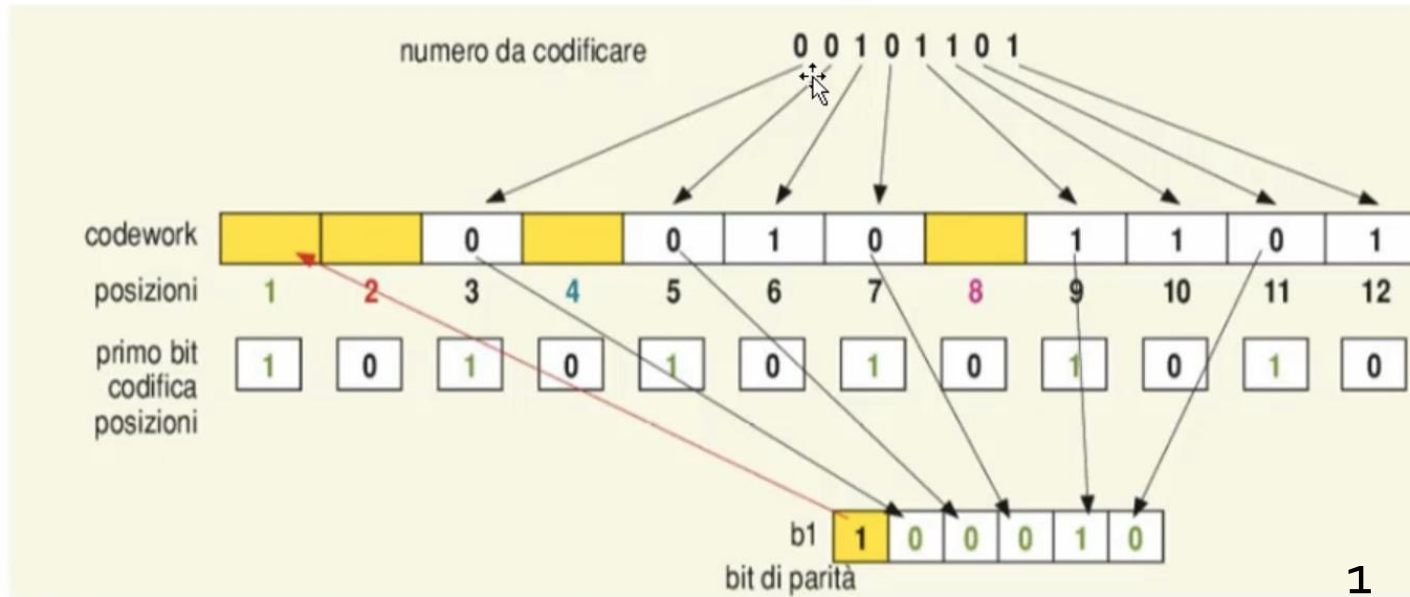
1+null=1 1+0=1 0+1=1

Dunque Gray: 111



Codici NON pesati Hamming

Codice Hamming - Esempio/codifica



Se gli 1 sono pari si mette zero in posizione, se gli 1 sono dispari si mette 1

- 1: 1,3,5,7,9,11
- 2: 2,3,6,7,10,11
- 3: 4,5,6,7,12
- 4: 8,9,10,11,12

- 1 si e 1 no
- 2 si e 2 no
- 4 si e 4 no
- 8 si e 8 no

100001011101



RIeseguiamo per la verifica di **ricezione** per ogni bit il **controllo di parità** sempre contando gli 1 presenti:

- 1 bit A somma bit 1 3 5 7 9 11 = pari ok
- 2 bit A somma bit 2 3 6 7 10 11 = pari ok
- 4 bit A somma bit 4 5 6 7 12 = pari ok
- 8 bit A somma bit 8 9 10 11 12 = pari ok

Sommando le posizioni dei bit con check errato, cioè il secondo e il quarto, otteniamo la posizione del bit del messaggio che si è sporcato: in questo caso **non ci sono errori**.

Supponiamo di aver ricevuto il codice 00101101 e lo analizziamo per verificarne la correttezza dei bit:

- 1 bit A somma bit 1 3 5 7 9 11 = pari ok
- 2 bit A somma bit 2 3 6 7 10 11 = dispari ERROR = POSIZ 2
- 4 bit A somma bit 4 5 6 7 12 = dispari ERROR = POSIZ 4
- 8 bit A somma bit 8 9 10 11 12 = pari ok

Sommando le posizioni 2+4 si è **sporcato: il bit numero 6**.

EAN

L'**EAN-13**, come suggerisce il nome, rappresenta un codice di 13 cifre.

È un codice pluridimensionale dove sono presenti 4 diversi spessori di barre/spazi, ognuno multiplo del modulo unitario e ogni carattere del codice è codificato come usuale in modo binario, usando 7 moduli continui, ovvero sono significativi sia gli spazi che le barre.

- l'EAN-13, o "normale";
- l'EAN-8, o "ridotto";
- l'EAN-2, usato nei periodici assieme all'EAN-13;
- l'EAN-5, usato per la carta stampata.

l'ultimo numero è un **check digit** (codice di controllo) che viene calcolato col seguente algoritmo:

- si moltiplicano le prime 12 cifre del codice alternativamente per 1 e per 3;
- si sommano i valori ottenuti;
- la cifra di controllo è il più piccolo numero da aggiungere a questa somma per ottenere un multiplo di 10.

Il codice EAN-13

L'**EAN-13**, come suggerisce il nome, rappresenta un codice di 13 cifre.



1	2	3	4	5	6	7	8	9	10	11	12	13
nazione		codice ditta					codice articolo				check digit	

TPSIT

IL SISTEMA OPERATIVO

Prof. Della Vecchia P.

Ricapitolazione

Elementi del sistema operativo

- 1. **Kernel** È il cuore del sistema operativo. Gestisce la comunicazione tra applicativi con CPU, memoria e periferiche.
- 2. **File System** Organizza e gestisce file e cartelle sul disco (percorsi).
- 3. **Memoria Virtuale** Estende la memoria RAM usando il disco rigido come supporto temporaneo. Permette ai processi di avere più memoria casuale disponibile di quella fisica reale.
- 4. **Scheduler** Decide quale processo deve usare la CPU e per quanto tempo. Garantisce l'efficienza e la correttezza nell'esecuzione concorrente dei processi.
- 5. **Spooler** Gestisce e ordina le code di stampa o di input/output.
- 6. **Shell** (interfaccia utente) È l'interfaccia tra utente e sistema operativo. Può essere testuale (prompt dei comandi, cli, cmd, terminale) o grafica (GUI).

IL SISTEMA OPERATIVO

Avvio del calcolatore BOOTSTRAP (Boot)

Programma in binario situato nel firmware che avvia le elaborazioni.

1. POST (Power On Self Test)

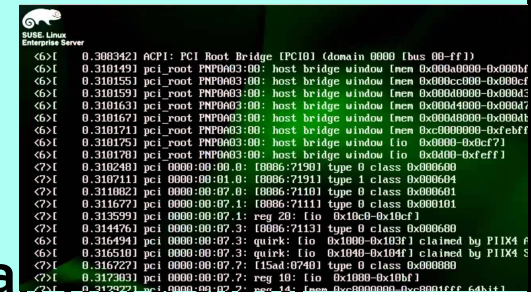
All'accensione, ogni componente hardware esegue un programma di autodiagnostica nel firmware. Controlla che tutti i dispositivi funzionino correttamente.

2. IPL (Initial Program Loader)

Nei processori a 32 bit, carica la prima istruzione di avvio all'indirizzo esadecimale 0xFFFFFFFF della RAM e carica sempre in Ram il bootloader.

Successivamente parte il bootloader: Caricamento del kernel

Viene caricata in RAM la parte principale del sistema operativo (kernel) da



```
0.308342] ACPI: PCI Root Bridge (PC10) (domain 0000 [bus 00-ff])
0.310149] pci_root PNP0A03:00: host bridge window [mem 0x000a0000-0x000bf
0.310155] pci_root PNP0A03:00: host bridge window [mem 0x000cc000-0x000cf
0.310159] pci_root PNP0A03:00: host bridge window [mem 0x000d0000-0x000d4
0.310163] pci_root PNP0A03:00: host bridge window [mem 0x000d4000-0x000d4
0.310167] pci_root PNP0A03:00: host bridge window [mem 0x000d4000-0x000d4
0.310171] pci_root PNP0A03:00: host bridge window [mem 0xc0000000-0xc0000
0.310175] pci_root PNP0A03:00: host bridge window [io 0x0000-0x0cf7]
0.310179] pci_root PNP0A03:00: host bridge window [io 0x0400-0xefff]
0.310249] pci 0000:00:00.0: [8086:7190] type 0 class 0x000600
0.310711] pci 0000:00:01.0: [8086:7191] type 1 class 0x000604
0.311082] pci 0000:00:07.0: [8086:7110] type 0 class 0x000601
0.311677] pci 0000:00:07.1: [8086:7111] type 0 class 0x000101
0.313599] pci 0000:00:07.1: reg 20: [io 0x10cf-0x1bcf]
0.314476] pci 0000:00:07.2: [8086:7113] type 0 class 0x000600
0.316494] pci 0000:00:07.3: quirk: [io 0x1000-0x103f] claimed by PIIX4
0.316510] pci 0000:00:07.3: quirk: [io 0x1040-0x104f] claimed by PIIX4
0.316727] pci 0000:00:07.7: [15ad:0740] type 0 class 0x000000
0.317203] pci 0000:00:07.7: reg 10: [io 0x1000-0x100f]
0.317292] pci 0000:00:07.7: reg 12: [mem 0xc0000000-0xc000ffff] 64KB
```

Post (Firmware)-> IPL (Firmware) ->Bootstrap (Firmware)-> Kernel (Hard disk)

Software di base

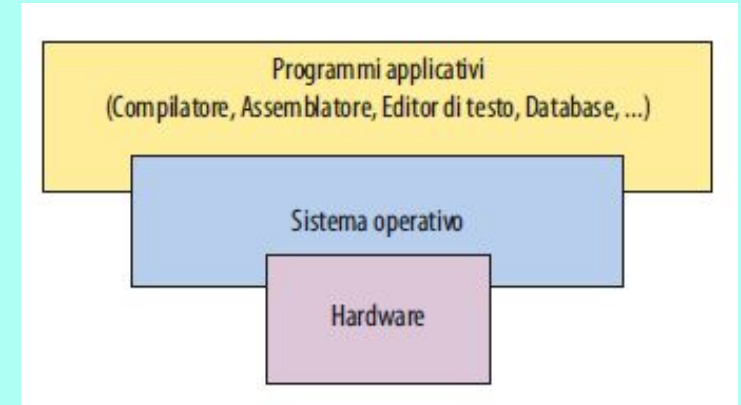
Il SO fa parte del **software di base**, termine con il quale si intende **l'insieme dei programmi** che consentono a un utente di eseguire **operazioni base** (**SO, Driver, utility di diagnostica**) come costruire e mandare in esecuzione un programma.

Il **software di base** è costituito da:

- il sistema operativo;
- gli editor di testo;
- i traduttori;
- i linker;
- i loader;
- i debugger.

Il sistema operativo svolge principalmente due compiti:

- è il **gestore delle risorse hardware** (CPU, memoria, periferiche) che vengono usate da specifici programmi per eseguire il proprio compito;
- fornisce il supporto **all'utente per impartire i comandi** necessari al funzionamento del computer, cioè fa da **interfaccia tra l'hardware e il software applicativo**.

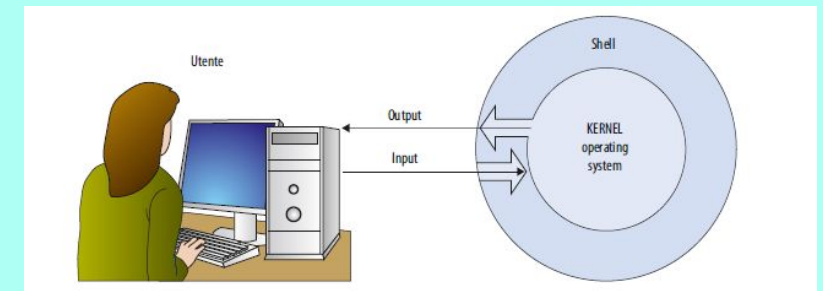


Il SO risiede sull'hard disk, solo il kernel che viene sempre caricato in memoria RAM.

KERNEL

Con kernel (nucleo) si intende il nucleo del sistema operativo che **avvolge idealmente tutto l'hardware**, partendo dalla CPU fino ai suoi dispositivi fisici, e si occupa di **interagire con i programmi applicativi (fa comunicare programmi applicativi con le risorse hardware)** che, ogni volta che necessitano di un servizio da parte di un dispositivo hardware, devono "passare attraverso lui" le **operazioni** che vengono eseguite dal kernel sono:

- avvio e terminazione dei programmi;
- assegnazione della CPU ai diversi processi;
- sincronizzazione tra i processi, è l'esecutore che segue lo scheduler. Lo scheduler decide, il kernel esegue.



LA GESTIONE DEI PROCESSI - SCHEDULER

Introduzione al multitasking

Tutti i moderni SO cercano di **sfruttare al massimo le potenzialità di parallelismo fisico** dell'hardware per **minimizzare i tempi di risposta** e aumentare il **throughput** (job per tempo) del sistema, ossia **quanti processi** il sistema riesce a completare in **un certo intervallo di tempo**.
un **job** nei processi è **un'unità di lavoro che il sistema operativo deve eseguire**

L'ottimizzazione del tempo di CPU avviene grazie **alla multiprogrammazione**, cioè alla **contemporanea** presenza di più programmi in memoria.

Differenziamo:

- **JOB SCHEDULER** che consiste nei processi **utilizzati dal S.O.** per la **scelta dei programmi** che dal disco **devono essere caricati in RAM**;
- **CPU SCHEDULER**, che consiste nei processi che permettono di **assegnare e sospendere** l'utilizzo della CPU da parte dei vari programmi.

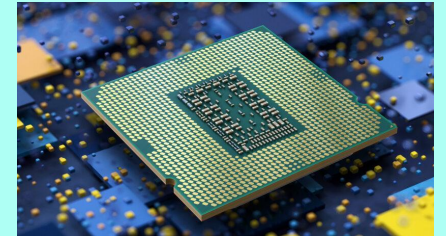
LA GESTIONE DEL PROCESSORE

I sistemi operativi multitasking durante le fasi di **attesa** mandano in esecuzione sulla CPU altri programmi tra quelli caricati in memoria, “**portando avanti**” in **parallelo più processi**, **riducendo al minimo l’inattività della CPU** e migliorando così l’efficienza del sistema (numero di programmi eseguiti per unità di tempo).

Il **parallelismo** che si ottiene è **virtuale (e non fisico)** dato che il processore è uno solo.

Il **parallelismo può essere:**

- **multitasking**: esecuzione di programmi in parallelismo alternato su singola CPU;
- **multiprocessing**: multiprogrammazione estesa a elaboratori dotati di **più CPU o più core**.
Un **core** è l’**unità di elaborazione interna** a un processore che esegue istruzioni in modo indipendente; ogni core funziona come una **piccola CPU** completa dentro il chip

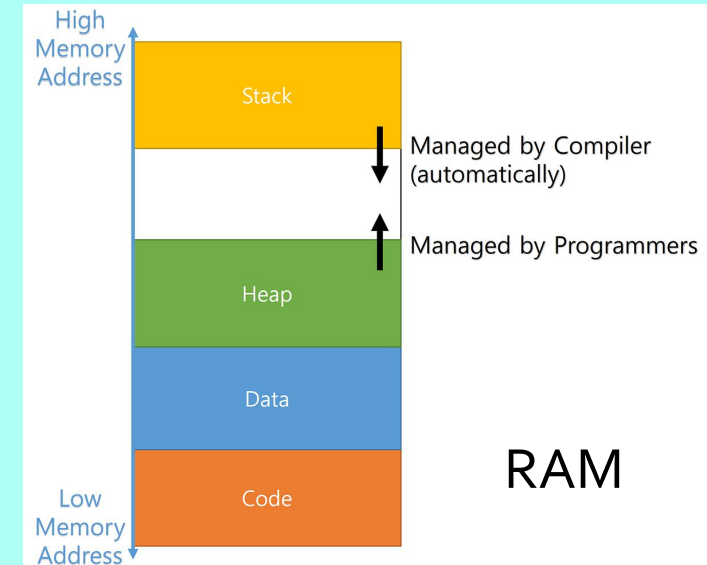


I processi

Come abbiamo visto, il **programma** è un'entità **passiva** (un insieme di byte contenente le istruzioni che dovranno essere eseguite) mentre il **processo** è un'entità **attiva**, che evolve man mano che le istruzioni vengono eseguite dalla CPU.

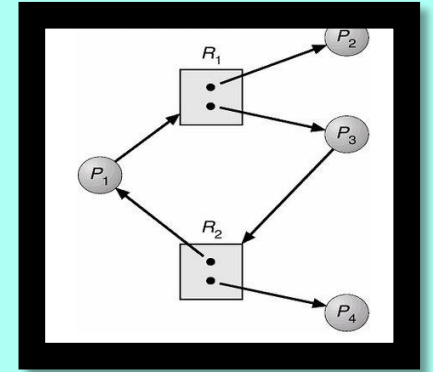
I **moderni processi** del sistema operativo possono essere costituiti da **due parti**:

- **il codice** (composto dalle istruzioni);
- **i dati del programma**, a loro volta suddivisi in:
 - **variabili globali**, allocate in memoria centrale nell'area dati globali **DATA** (RAM);
 - **variabili locali e non locali** delle procedure del programma, memorizzate in uno **stack**;
 - **variabili temporanee introdotte dal compilatore** caricate nei **registri del processore**;
 - **variabili allocate dinamicamente** durante l'esecuzione, memorizzate in un **heap**.



PROCESSI SEQUENZIALI E PARALLELI

Risorse: CPU e calcolo, spazio in Ram, spazio su disco, periferiche i/o, risorse di rete, variabili di sistema, semafori



- ▶ I processi in esecuzione possono essere indipendenti oppure cooperare per raggiungere un medesimo obiettivo:
 - **processi indipendenti**, un processo evolve in modo autonomo senza bisogno di comunicare con gli altri processi per scambiarsi i dati;
 - **processi paralleli o cooperanti** hanno la necessità di cooperare in quanto, per poter evolvere, hanno bisogno di scambiarsi informazioni o meglio risorse.

Oltre alla cooperazione esiste un'altra forma di interazione tra processi:

- **Processi in competizione**, ovvero quando due (o più) processi che si ostacolano a vicenda compromettendo il buon fine delle loro elaborazioni generano un **blocco individuale o critico (deadlock)**.

È il caso in cui entrambi i processi competono per utilizzare la medesima **risorsa**, che magari è in quantità *limitata nel sistema*.

STATO DI UN PROCESSO

In che stato può trovarsi **un processo rispetto al processore?**

- **NUOVO (new, init):** stato di un processo appena creato, cioè l'utente richiede l'esecuzione di un programma ed è pronto ad andare in **RL (Ready List)**;
- **PRONTO (ready-to-run):** se ha tutte le risorse necessarie alla sua evoluzione e riceve l'input di assegnazione dalla cpu.
- In **ESECUZIONE (running):** il processo sta evolvendo, la CPU sta eseguendo le sue istruzioni;
- In **ATTESA (waiting o blocked):** quando gli manca una risorsa per poter evolvere il processo si sospende (**Suspend**) e quindi aspetta(in **Waiting List WL**)_che si verifichi un evento di un altro processo o del processore; dopodichè torna in **Ready List**;
- **FINITO (terminated):** siamo nella situazione in cui tutto il codice del processo è stato eseguito e quindi ha determinato l'esecuzione; il sistema operativo deve ora rilasciare le risorse che utilizzava.

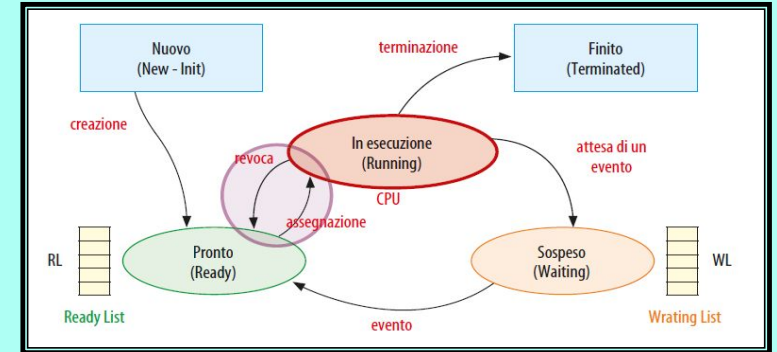
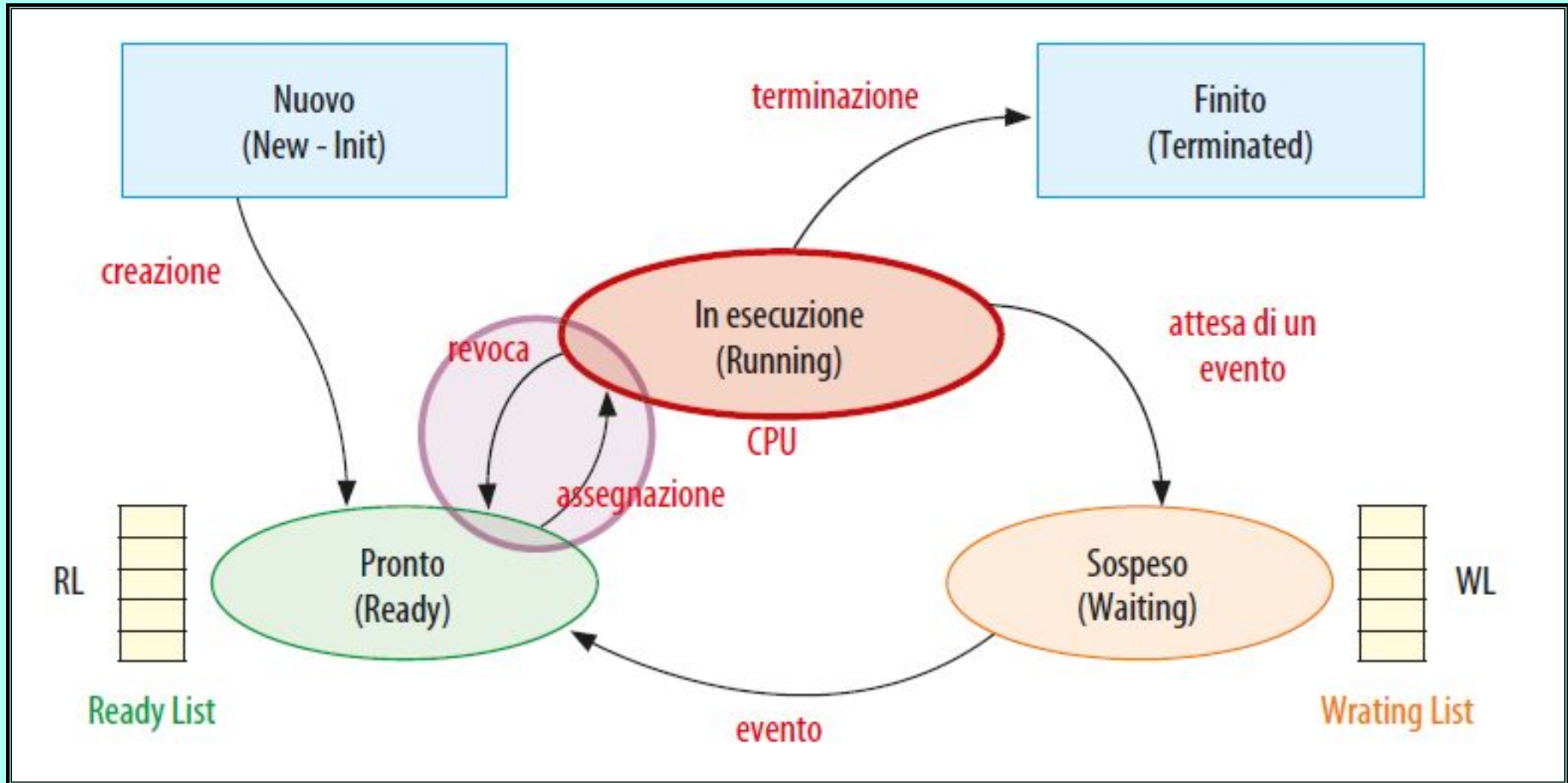


DIAGRAMMA DEGLI STATI



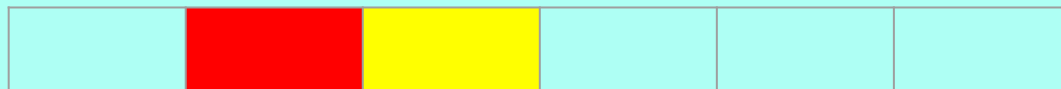
I processi

Lo stato di un processo prende anche il nome di **contesto del processo** che, naturalmente, **varia istante per istante**, a partire dal valore contenuto nel **program counter**, un **registro della CPU**

Alcuni registri DELLA CPU:

Il **program counter (PC)** è un registro interno della CPU che contiene le **istruzioni** della **prossima** istruzione da eseguire. **Punta, incrementa e salta.**

IR Instruction register è un altro registro nel processore e contiene l'**indirizzo** di un'istruzione del processo che in quel momento è in esecuzione.

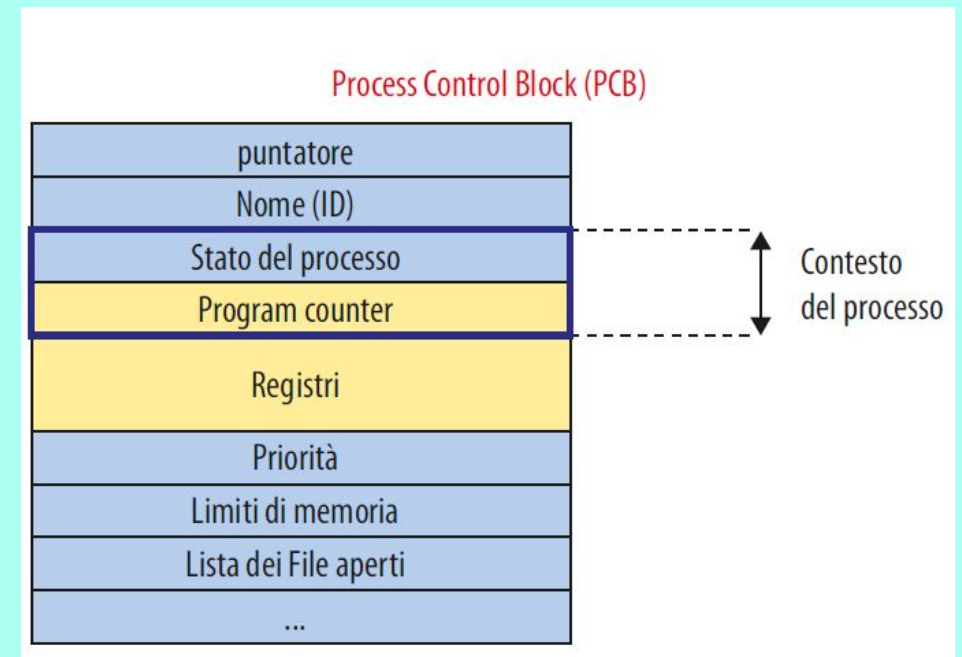


Quando un processo cambia stato si parla di **cambio di contesto (context-switch)**.
La parte del SO che realizza il cambio di contesto si chiama **dispatcher, una parte del kernel**.

Il **contesto di un processo** è composto da alcune informazioni (STATO E ProgramCounter) contenute nel suo specifico **PCB (Process Control Block)** che è utile a CPU e SO per capire in che punto si trova un processo

Il PCB CONTIENE:

- **identificatore unico (PID);**
- **stato corrente;**
- **program counter;**
- **registri;**
- **priorità;**
- **puntatori alla memoria del processo;**
- **puntatori alle risorse allocate al processo.**



Pre-emptive

Processi che possono essere interrotti anche se non hanno terminato la loro esecuzione

Non Pre-emptive

Non tutti i processi possono essere sospesi dal SO in ogni istante della loro esecuzione: per loro natura alcuni processi **devono terminare la loro esecuzione** (oppure un insieme di istruzioni che devono **essere eseguite senza interruzione**, come un'operazione di I/O) e solo quando sono in particolari situazioni possono essere interrotti.

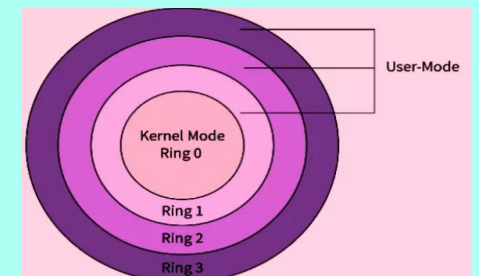
I sistemi a divisione di tempo (time sharing) hanno uno scheduling pre-emptive. Ad esempio la suddivisione di una larghezza di banda ogni secondo.

User mode e kernel mode

Durante la loro attività i processi, si **alternano** anche in **diverse modalità di esecuzione**, caratterizzate da diversi livelli di **privilegio**:
il **livello user mode** e il **livello kernel mode**.

Il livello **user mode** è quello di “**normale stato di esecuzione**” dei **programmi applicativi** dell’utente,
mentre il livello **kernel mode**, detto anche supervisore, è quello nel quale sono in esecuzione i **servizi del kernel**.

Quando il processore è in **kernel mode**, cioè sta eseguendo un programma in modalità kernel, vengono opportunamente **settati alcuni bit di stato del processore** e in questo stato, generalmente, la sua esecuzione non può essere interrotta, **comprometterebbe la cpu**: questi processi sono quindi **sempre non pre-emptive**.



Throughput

Il numero medio di job, programmi, processi o richieste completati dal sistema nell'unità di tempo.

Tempo di completamento (turnaround time)

Il tempo dalla sottomissione di un job, programma o processo da parte di un utente nel momento in cui i **risultati sono resi effettivamente disponibili all'utente stesso**. (Caricamento)

Tempo di risposta (response time)

Il tempo dalla sottomissione di una richiesta da parte dell'utente nel momento in cui il processo risponde, **chiamato anche tempo di latenza**.

Tempo di attesa (wait time)

Si ottiene dalla somma degli intervalli temporali passati in attesa della risorsa.

Tempi di Burst di CPU

Uso continuativo della CPU da parte di un processo

Tempi di Burst di I/O

Uso continuativo di un dispositivo di I/O da parte di un processo

Algoritmi di scheduling

Processo__	Arrivo__	Attesa	Priorità
P ₁	0	8 +	2
P ₂	1	4 +	1
P ₃	2	2	3

- **Algoritmo di scheduling FC_FS**

Esegue in **ordine di arrivo**. Nessuna priorità reale. Non preemptive.

P = attesa effettiva - arrivo

$$P_1 = 8 - 0 = 8$$

$$P_2 = 12 - 1 = 11$$

$$P_3 = 14 - 2 = 12$$

p - attesa

$$At_1 = 8 - 8 = 0$$

$$At_2 = 11 - 4 = 7$$

$$At_3 = 12 - 2 = 10$$

$$At_{medio} = (0 + 7 + 10) / 3 = 5,67$$

- **Algoritmo di scheduling SJ_F**

Sceglie il processo con **burst time minore** tra quelli arrivati. Priorità attesa più corta. Non preemptive.

Al primo giro parte per forza p1 e poi con priorità quindi P1, P3 e P2

attesa effettiva - arrivo

$$P_1 = 8 - 0 = 8$$

$$P_3 = 10 - 2 = 8$$

$$P_2 = 14 - 1 = 13$$

p - attesa

$$At_1 = 8 - 8 = 0$$

$$At_3 = 8 - 2 = 6$$

$$At_2 = 13 - 4 = 9$$

$$At_{medio} = (0 + 6 + 9) / 3 = 5$$

- **Scheduling con priorità**

Esegue il processo **con priorità** numericamente più alta (numero più basso).

Non preemptive. **Al primo giro parte per forza p1 e poi con priorità quindi p1,p2,p4,p3**
attesa effettiva - arrivo

$$P_1 = 8 - 0 = 8$$

$$P_2 = 12 - 1 = 11$$

$$P_4 = 14 - 3 = 11$$

$$P_3 = 16 - 2 = 14$$

p - attesa

$$At_1 = 8 - 8 = 0$$

$$At_2 = 11 - 4 = 7$$

$$At_4 = 11 - 2 = 9$$

$$At_3 = 14 - 2 = 12$$

$$At_{medio} = (0+7+9+12)/4 = 7$$

- **Algoritmo di scheduling Round Robin**

Ogni processo riceve una fetta di CPU (quantum ad esempio di 3). Priorità equità temporale.

p1 3(resta 5), p2 3 (resta 1), p3 3 (finito), p4 3 (finito), p1 3 (resta 2), p2 3 (finito), p1 3 (finito)

attesa effettiva - arrivo

$$P_1 = 16 - 0 = 16$$

$$P_2 = 14 - 1 = 13$$

$$P_4 = 12 - 3 = 9$$

$$P_3 = 8 - 2 = 6$$

p - attesa

$$At_1 = 16 - 8 = 8$$

$$At_2 = 13 - 4 = 9$$

$$At_4 = 9 - 2 = 7$$

$$At_3 = 6 - 2 = 4$$

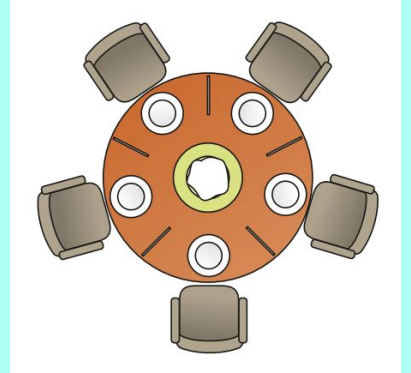
$$At_m = (8+9+7+4)/4 = 7$$

Processo__	Arrivo__	Attesa	Priorità
P1	0	8 +	2
P2	1	4 +	1
P3	2	2	4
P4	3	2 +	3

Problemi di sincronizzazione deadlock

I processi **indipendenti** possono girare su computer diversi.

I processi **cooperanti** devono condividere **risorse**, anche solo per comunicare.



Esempio: i filosofi a cena (Dijkstra, 1965)

Cinque filosofi, cinque piatti, cinque bacchette.
Per mangiare, servono **due bacchette**.

Se tutti prendono **una bacchetta**, rimangono bloccati: **nessuno può prendere la seconda**.

Soluzioni di base

Variabili condivise (share variables)

Usate quando i processi condividono la **stessa memoria**.

Scambio di messaggi (message passing)

Permette comunicazione anche tra processi su **computer diversi**.